

CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

ARRAY LIST & LINKED LIST



TS. TRẦN NGỌC VIỆT
TH.S LÊ CÔNG HIẾU



HỌC KỲ 3 – NĂM HỌC 2025-2026



KHÓA K30 & K31



1. Danh sách mảng

Khái niệm cấu trúc dữ liệu

- Kiểu dữ liệu (data-type) là kiểu lưu trữ dữ liệu mà ngôn ngữ máy tính sẽ cho phép chẳng hạn như số nguyên (int), dấu phẩy động (float, double), ký tự (char), v.v.
- Cấu trúc dữ liệu (data structure) là kiểu dữ liệu được xây dựng bởi lập trình viên để trừu tượng hóa sự phức tạp của các dữ liệu (thuộc tính) và các hoạt động của nó.



1. Danh sách mảng

Cấu trúc dữ liệu

- Cấu trúc dữ liệu là một mô hình toán học được đặc trưng bởi các thuộc tính sau:
 - Cấu trúc dữ liệu được xác định bởi một số dữ liệu và một tập hợp hoạt động, thao tác trên dữ liệu đó.
 - Các thao tác được sử dụng với các giao diện trực quan - các hoạt động chỉ có thể được truy cập thông qua giao diện.
 - Có thể xây dựng các tiên đề chính thức, điều kiện trước/sau vào kiểu dữ liệu và các hoạt động liên quan.



1. Danh sách mảng

Cấu trúc dữ liệu

- Cấu trúc dữ liệu phải độc lập với ngôn ngữ lập trình:
 - Tuy nhiên một số loại cấu trúc dữ liệu lại dễ triển khai ở một số ngôn ngữ này hơn những ngôn ngữ khác.
- Cấu trúc dữ liệu nên được triển khai độc lập với lĩnh vực ứng dụng:
 - Một số cấu trúc dữ liệu có thể không phù hợp với một số các loại miền và ứng dụng



1. Danh sách mảng

Cấu trúc dữ liệu

- Để xây dựng một cấu trúc dữ liệu đầy đủ, phải cần đưa ra những điều sau:
 - Mô tả các yếu tố, trạng thái, dữ liệu tạo nên cấu trúc dữ liệu và mô tả các mối quan hệ giữa các phần tử riêng lẻ trong nó.
 - Mô tả tất cả các hoạt động có thể được thực hiện trên các dữ liệu của cấu trúc dữ liệu.



1. Danh sách mảng

Cấu trúc dữ liệu mảng

Mảng -Array là một trong các cấu trúc dữ liệu thường gặp nhất. Mảng có thể lưu giữ một số phần tử cố định và các phần tử này có cùng kiểu. Cấu trúc dữ liệu đều sử dụng mảng để triển khai cấu trúc giải thuật.

-Phần tử: Mỗi mục được lưu giữ trong một mảng được gọi là một phần tử.

-Chỉ mục: Mỗi vị trí của một phần tử trong một mảng có một chỉ mục số được sử dụng để nhận diện phần tử.

Mảng gồm các bản ghi có kiểu giống nhau, có kích thước cố định, mỗi phần tử được xác định bởi chỉ số.

1. Danh sách mảng

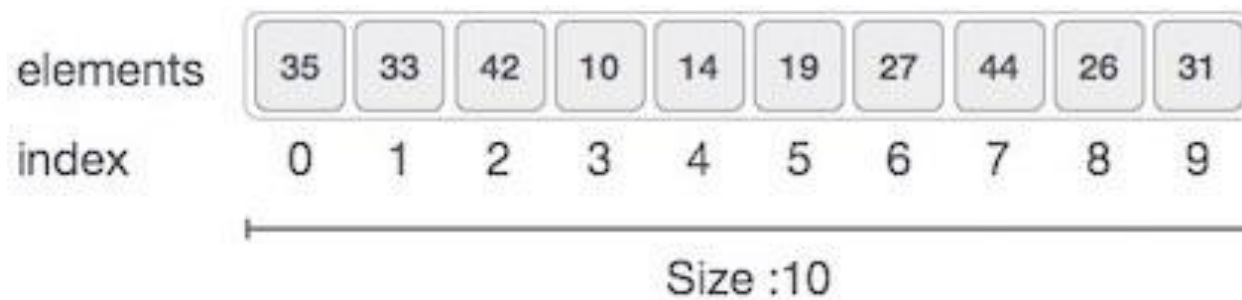
Biểu diễn Cấu trúc dữ liệu mảng

Mảng có thể được khai báo theo nhiều cách đa dạng trong các ngôn ngữ lập trình.

Minh họa: sử dụng phép khai báo mảng trong ngôn ngữ C:



Minh họa phần tử và chỉ mục:





1. Danh sách mảng

Một số điểm cần ghi nhớ cấu trúc dữ liệu mảng:

- Chỉ mục bắt đầu với 0.
- Độ dài mảng là 10, là mảng có thể lưu giữ 10 phần tử.
- Mỗi phần tử đều có thể được truy cập thông qua chỉ mục của phần tử đó.
- Ví dụ, chúng ta có thể lấy giá trị của phần tử tại chỉ mục 5 là 19.

Phép toán cơ bản về mảng như:

- **Duyệt:** In tất cả các phần tử mảng theo cách in từng phần tử một.
- **Chèn:** Thêm một phần tử vào mảng tại chỉ mục đã cho.
- **Xóa:** Xóa một phần tử từ mảng tại chỉ mục đã cho.
- **Tìm kiếm:** Tìm kiếm một phần tử bởi sử dụng chỉ mục hay bởi giá trị.



1. Danh sách mảng

Chèn phần tử vào mảng:

Hoạt động chèn là để chèn một hoặc nhiều phần tử dữ liệu vào trong một mảng. Tùy theo yêu cầu, phần tử mới có thể được chèn vào vị trí đầu, vị trí cuối hoặc bất kỳ vị trí chỉ mục đã cho nào của mảng.

Ví dụ: Giả sử A là một mảng tuyến tính không có thứ tự có n phần tử và k là một số nguyên dương thỏa mãn $k \leq n$. Dưới đây là giải thuật chèn phần tử a vào vị trí thứ k của mảng A.

1. Danh sách mảng

Giải thuật array list – chèn phần tử vào mảng

1. Bắt đầu
2. Gán $J=n$
3. Gán $n = n+1$
4. Lặp lại bước 5 và 6 khi $J \geq k$
5. Gán $A[J+1] = A[J]$
6. Gán $J = J-1$
7. Gán $A[k] = \text{ITEM}$
8. Kết thúc

```
main() {
    int A[] = {105,43,55,70,8,26};
    int item = 39, k = 4, n = 6;
    int i = 0, j = n;

    printf("Danh sach cua mang ban dau:\n");

    for(i = 0; i<n; i++) {
        printf("A[%d] = %d \n", i, A[i]);
    }

    n = n + 1;

    while( j >= k){
        A[j+1] = A[j];
        j = j - 1;
    }

    A[k] = item;

    printf("Danh sach cua mang sau khi chen:\n");

    for(i = 0; i<n; i++) {
        printf("A[%d] = %d \n", i, A[i]);
    }
}
```



1. Danh sách mảng

Lợi ích của cấu trúc dữ liệu

- Cung cấp quyền truy cập vào các loại đặc biệt cung cấp các dịch vụ chuyên biệt
- Dễ dàng sử dụng các dịch vụ bằng cách đóng gói phức tạp bằng cách ẩn dữ liệu và hoạt động đằng sau mặt tiền của một hoặc nhiều giao diện.
- Thúc đẩy tái sử dụng và giảm thời gian phát triển - dễ dàng sử dụng lại các dịch vụ phức tạp của nó.
- Thúc đẩy tập trung cải tiến giải thuật của chương trình (đặc biệt là hoạt động phức tạp) - các thuộc tính khác nhau có thể được lấy từ các thông số kỹ thuật chính thức và được xây dựng vào cấu trúc dữ liệu.



1. Danh sách mảng

Lợi ích của cấu trúc dữ liệu

- Cải thiện khả năng tái sử dụng mã.
- Giảm sự phức tạp và thực hiện các hoạt động phức tạp đơn giản cho lập trình viên
- Thực hiện triển khai thực tế đơn giản, dễ hiểu.



1. Danh sách mảng

Tiếp theo chúng ta tìm hiểu:

- Mảng đóng vai trò danh sách.
- Các cách hoạt động trên arraylist



1. Danh sách mảng

Danh sách

- Thông thường lưu trữ thông tin trong danh sách: như danh sách lớp lưu thông tin sinh viên...
- Danh sách này là cấu trúc dữ liệu cơ bản nhất và được sử dụng để thiết kế và thực hiện cấu trúc dữ liệu phức tạp hơn.



1. Danh sách mảng

Danh sách

- Danh sách có thể là danh sách rỗng hoặc có chứa nhiều phần tử
 - List $j = a_1, a_2, a_3, \dots, a_n$
- Các phần tử được sắp xếp tuyến tính theo vị trí của nó trong danh sách sao cho:
 - a_1 rồi tới a_2 , a_2 rồi tới a_3 , a_3 rồi tới a_4 ...cứ như vậy a_i rồi tới a_{i+1}
 - Phần tử đầu tiên là a_1 , Phần tử cuối cùng là a_n
 - Số phần tử trong danh sách là n cũng là chiều dài của danh sách.
 - Nếu $n=0$ thì danh sách rỗng.



1. Danh sách mảng

Các thao tác trên danh sách

- Append Element: đưa 1 phần tử mới vào danh sách
- Insert Element: chèn 1 phần tử vào giữa danh sách
- Remove Element: xóa 1 phần tử, 1 giá trị khỏi danh sách.
- Get Element: lấy giá trị của phần tử
- Find Element: tìm kiếm phần tử
- Swap Elements: đảo giá trị hai phần tử
- Go to End of List: về cuối danh sách
- Go to Head of List: lên đầu danh sách
- Go to Next Element: truy xuất phần tử tiếp theo
- Clear List: xóa danh sách.



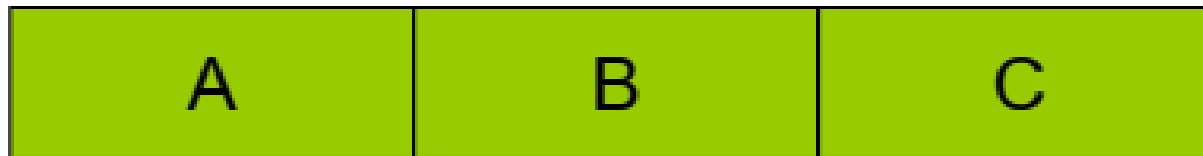
1. Danh sách mảng

Các ví dụ thao tác

- Thêm 1 phần tử vào danh sách
- *Append(A,L)* : Thêm giá trị A vào cuối danh sách L
- *Append(B,L)* : Thêm giá trị B vào cuối danh sách L
- *Append(C,L)* : Thêm giá trị C vào cuối danh sách L

Before Operations L

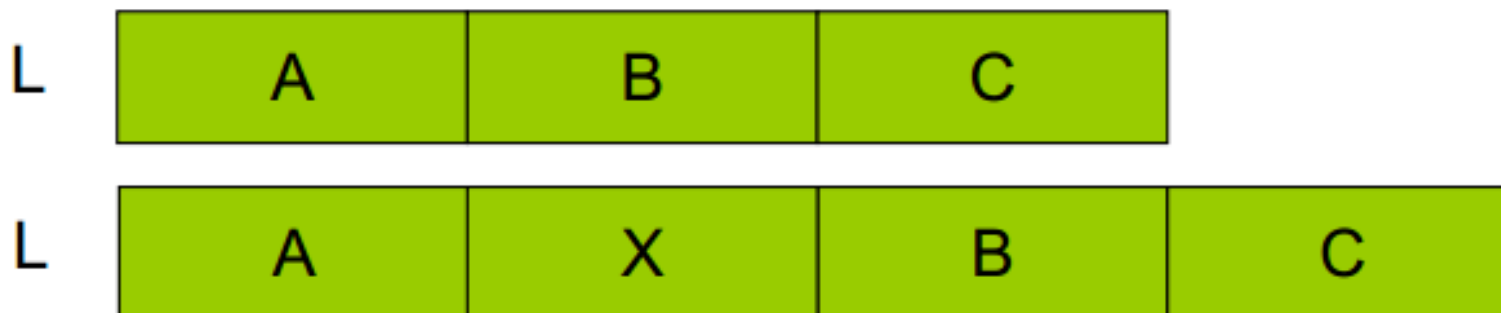
After Operations L



1. Danh sách mảng

Chèn 1 phần tử

- $Insert(1, X, L)$: Chèn 1 giá trị X vào danh sách L, tại vị trí số 1
- Ta thấy danh sách L trước và sau khi chèn X



1. Danh sách mảng

Sửa đổi giá trị một phần tử

- $Set(2, Z, L)$: cập nhật giá trị Z tại vị trí số 2 trong L

L	A	X	B	C
---	---	---	---	---

L	A	X	Z	C
---	---	---	---	---



1. Danh sách mảng

Xóa một phần tử

- *Remove(Z,L)* : xóa giá trị Z trong L
- Trường hợp 1: xóa Z đầu tiên trong L

L	A	X	Z	Z
---	---	---	---	---

L	A	X	Z
---	---	---	---

1. Danh sách mảng

Xóa tất cả một phần tử có giá trị z

- *RemoveAll(Z,L)* : xóa tất cả Z trong L

L	A	X	Z	Z
---	---	---	---	---

L	A	X
---	---	---

1. Danh sách mảng

Lấy giá trị tại một vị trí theo yêu cầu

- *Get(2,L)* : Lấy giá trị tại vị trí thứ 3 của danh sách (B)

L	A	X	B	C
---	---	---	---	---

1. Danh sách mảng

Tìm kiếm vị trí một phần tử có giá trị X

- *Find(X,L)* : Tìm giá trị X trong danh sách, kết quả trả về vị trí = 1

L	A	X	B	C
---	---	---	---	---

1. Danh sách mảng

Hoán đổi 2 phần tử

- *Swap(0,3): Hoán đổi giá trị của 2 vị trí trong danh sách (vị trí số 0 và 3)*

L	A	X	B	C
L	C	X	B	A



1. Danh sách mảng

Ân các thao tác

- Để thực hiện các chức năng danh sách cơ bản này, chúng tôi sẽ cần một số chức năng riêng tư (private)
 - tạo / khởi tạo danh sách
 - xem qua danh sách
 - tìm các vị trí cụ thể trong danh sách
 - đọc các phần tử từ danh sách
 - biểu thị phần đầu và phần cuối của danh sách
 - • ...và rất nhiều thao tác con khác
- Hầu hết các hoạt động này sẽ không được tương tác với người dùng của cấu trúc dữ liệu danh sách.



1. Danh sách mảng

Cách hiện thực cấu trúc dữ liệu

- Có hai cách để thực hiện cấu trúc dữ liệu danh sách:
 - mảng
 - danh sách liên kết
- Trong hầu hết các ngôn ngữ, mảng là một cấu trúc, đó là kích thước của chúng được thiết lập tại thời điểm biên dịch.
- Danh sách được liên kết là động, tức là chúng có thể phát triển và thu nhỏ trong thời gian chạy - hơn thế nữa sau này...



1. Danh sách mảng

Triển khai danh sách mảng

Thuộc tính danh sách mảng:

- Vị trí của mỗi phần tử được cho bởi một chỉ mục từ 0 đến $n-1$, trong đó n là số các dữ liệu.
- Cho bất kỳ chỉ mục nào, phần tử có chỉ mục đó có thể được truy cập trong thời gian không đổi, ví dụ $O(1)$.
- Để thêm một phần tử vào cuối danh sách, thời gian thực hiện không phụ thuộc vào kích thước của danh sách.



1. Danh sách mảng

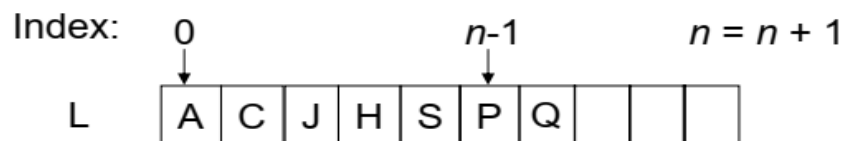
Triển khai danh sách mảng

- Thao tác chèn trong danh sách mảng phụ thuộc vào kích thước danh sách:
 - các vị trí phải được dịch chuyển.
 - có khả năng mảng phải được thay đổi kích thước
 - phần chèn gần đầu danh sách mất nhiều thời gian hơn phần chèn gần giữa hoặc cuối
- Thao tác xóa phụ thuộc vào kích thước của danh sách:
 - sau khi xóa một phần tử, các phần tử tiếp theo phải được chuyển xuống
 - các lần xóa gần đầu danh sách mất nhiều thời gian hơn xóa gần giữa hoặc cuối

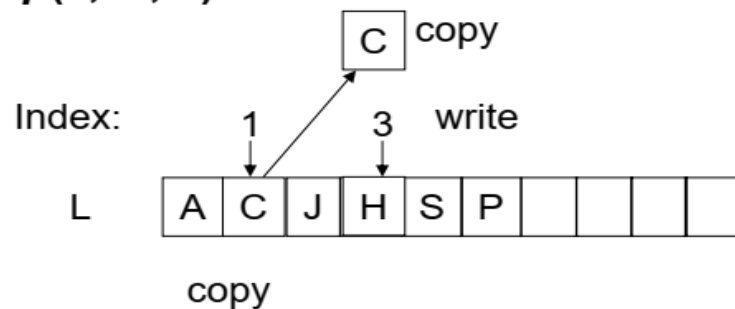
1. Danh sách mảng

Array List Operations – 1

Appending is trivial: Assume we want to add Q to the list or **Append(Q, L)**



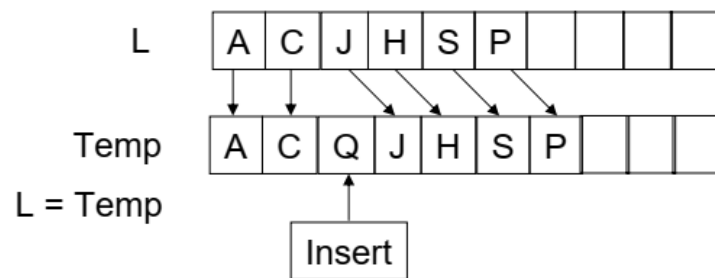
Swapping elements is also trivial: Assume we want to swap C and H or **Swap(1, 3, L)**



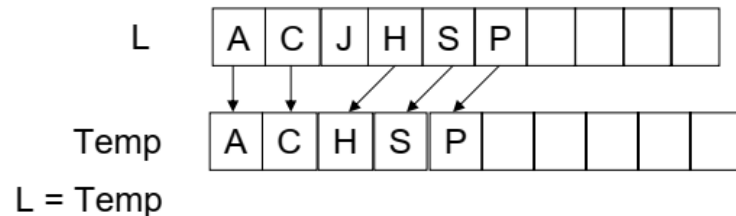
1. Danh sách mảng

Array List Operations – 2

Insertion is costly: Assume we want to insert Q in element 3 or ***Insert(Q, 3, L)***



Deletion is also costly: Assume delete J in element 3 or ***Delete(2, L)***



1. Danh sách mảng

Chỉ mục trong mảng

- Các ngôn ngữ hỗ trợ mảng có lập chỉ mục cơ chế cho phép lập trình viên dễ dàng truy cập các phần tử của mảng

0	1	2	3	4	5	...
'r'	'd'	'j'	'q'	's'	'l'	...

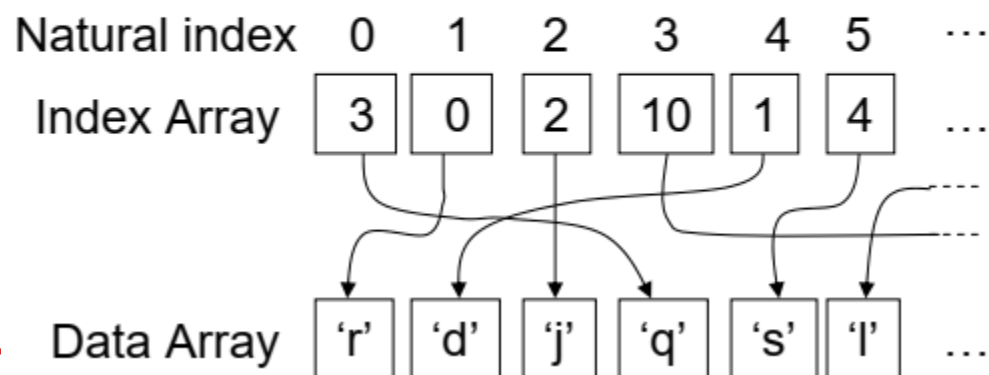
- Đôi khi có thể hữu ích khi phát triển chiến lược lập chỉ mục thay thế

1. Danh sách mảng

Định địa chỉ mảng gián tiếp

- Giải quyết gián tiếp liên quan đến việc sử dụng danh sách mảng để duy trì một danh sách các chỉ số vào một mảng dữ liệu khác
 - Cho phép sử dụng để duy trì chuyển tải danh sách khác nhau thứ tự rằng chỉ mục tự nhiên mà không cần sắp xếp lại danh sách dữ liệu

```
for i = 0 to arraysize  
{  
    print ( data[index[i]] );  
}
```



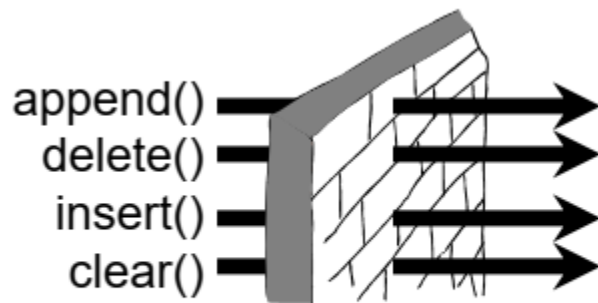
1. Danh sách mảng

Rõ ràng là nhiều thao tác trong số này hoạt động rất phức tạp

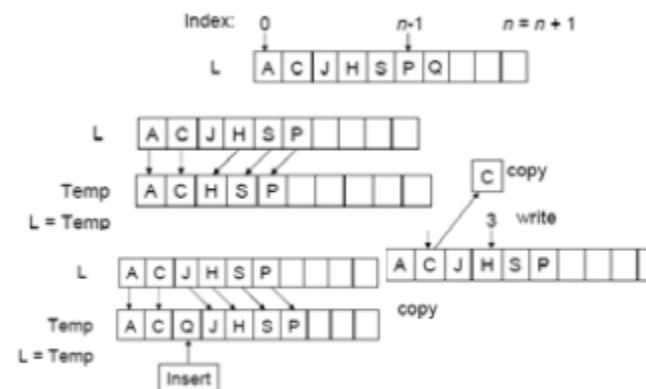
- Cấu trúc dữ liệu cho phép chúng tôi "che giấu mớ hỗn độn" và khuyến khích
 - tính đúng đắn - làm đúng một lần, dùng nhiều lần
 - khả năng dự đoán - hành vi được biết đến
 - năng suất - ít thời gian dành cho việc tái phát minh



Programmer



Application
Interface

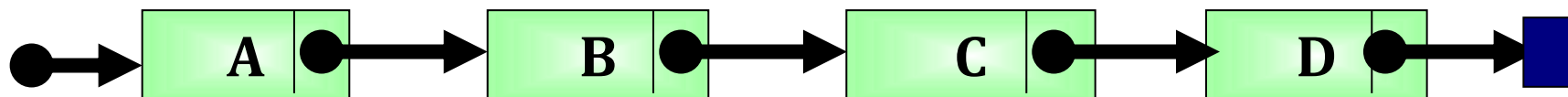


ADT

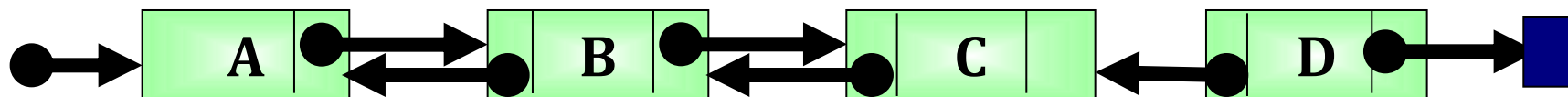
2. Danh sách liên kết

CÁC LOẠI DANH SÁCH LIÊN KẾT

Danh sách liên kết đơn: Mỗi phần tử liên kết với phần tử đứng sau nó trong danh sách



Danh sách liên kết kép: Mỗi phần tử liên kết với phần tử đứng trước và sau nó trong danh sách

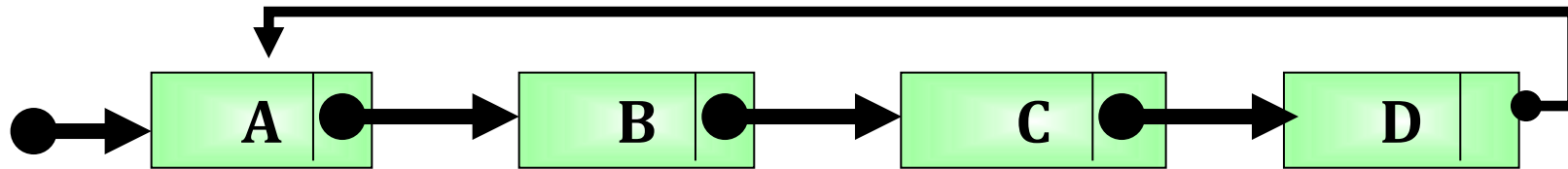


2. Danh sách liên kết

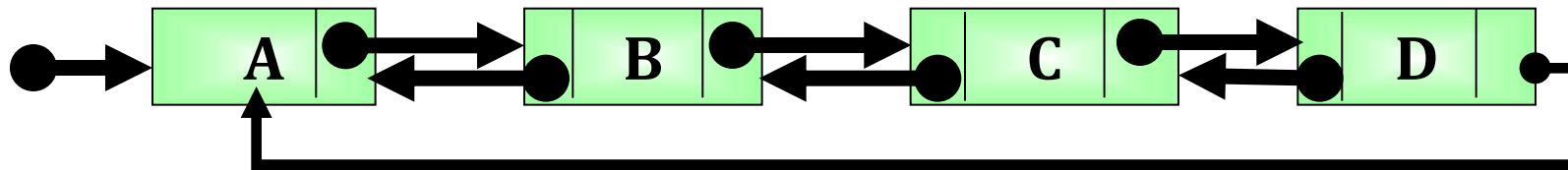
CÁC LOẠI DANH SÁCH LIÊN KẾT (TT)

Danh sách liên Vòng: Phần tử cuối danh sách liên với phần tử đầu danh sách

- Danh sách liên kết đơn vòng



- Danh sách liên kết đôi vòng

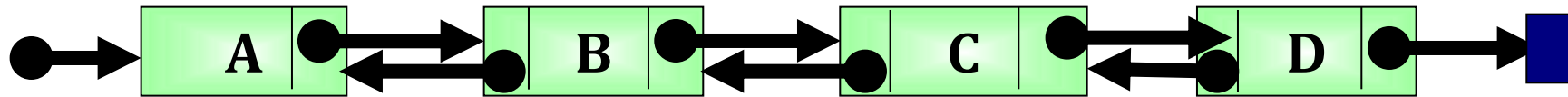


2. Danh sách liên kết

Danh sách liên kết kép

Mỗi phần tử liên kết với phần tử đứng trước và sau nó trong danh sách

Hình vẽ minh họa danh sách liên kết kép:





2. Danh sách liên kết

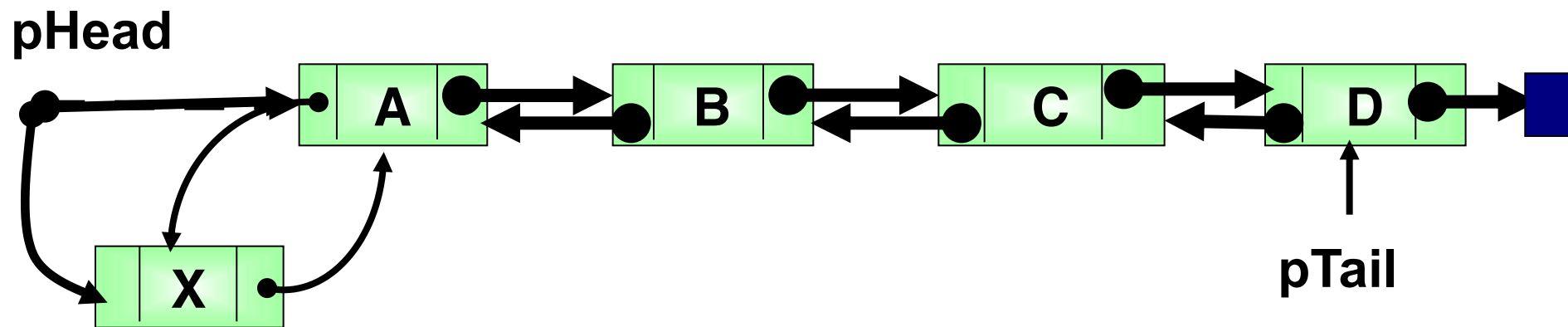
Các thao tác trên DSLK kép

- Khởi tạo danh sách liên kết kép rỗng
- Tạo 1 nút có thành phần dữ liệu = x
- Chèn 1 phần tử vào danh sách
 - Chèn vào đầu
 - Chèn sau phần tử Q
 - Chèn vào trước phần tử Q
 - Chèn vào cuối danh sách
- Hủy 1 phần tử trong danh sách
 - Hủy phần tử đầu danh sách
 - Hủy phần tử cuối danh sách
 - Hủy 1 phần tử có khoá bằng x
- Tìm 1 phần tử trong danh sách
- Sắp xếp danh sách

2. Danh sách liên kết

Thêm 1 nút vào đầu danh sách

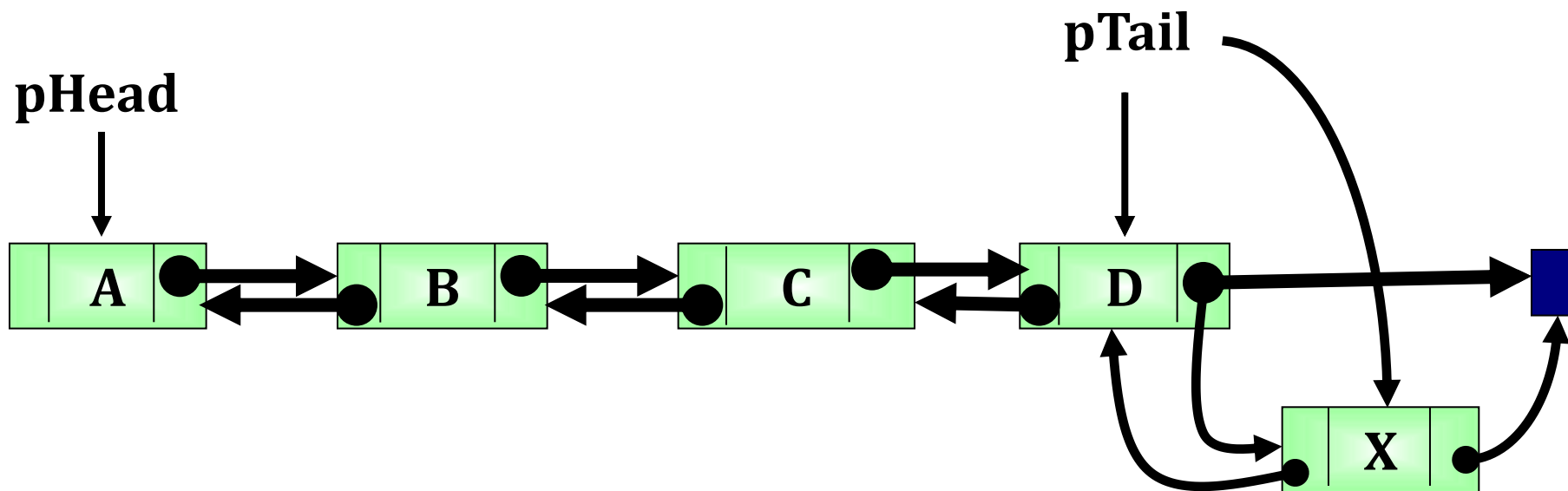
- Minh họa hình vẽ



2. Danh sách liên kết

Thêm vào cuối danh sách

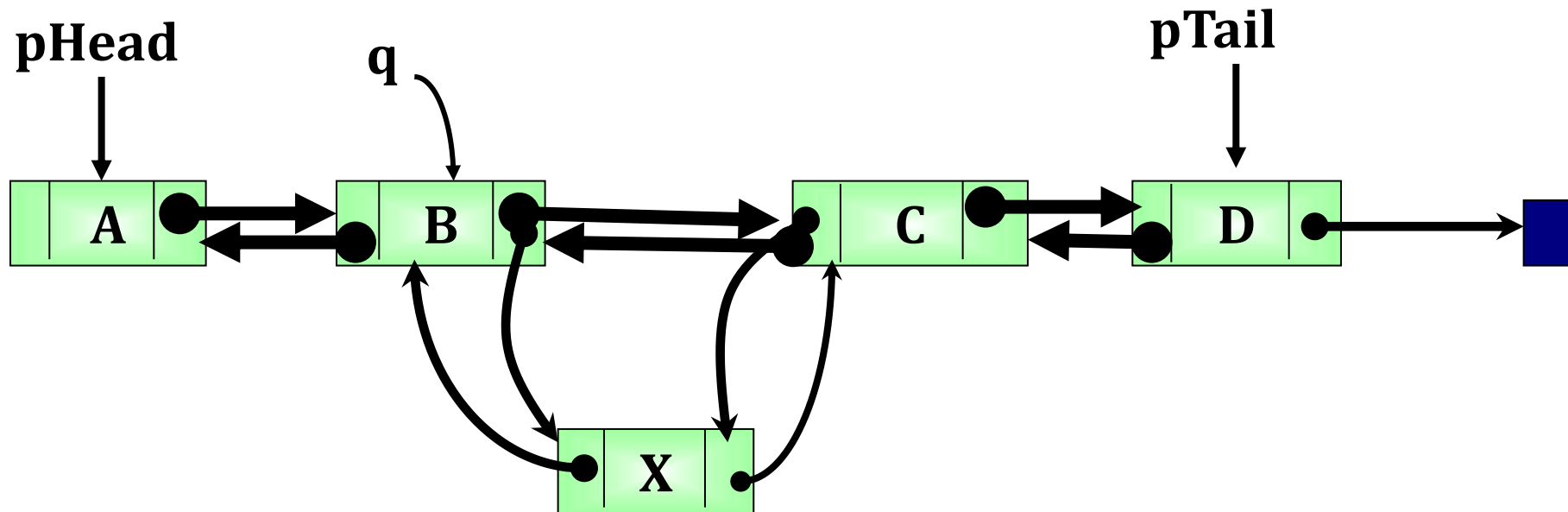
- Minh họa thêm 1 phần tử vào sau danh sách



2. Danh sách liên kết

Thêm vào sau nút q

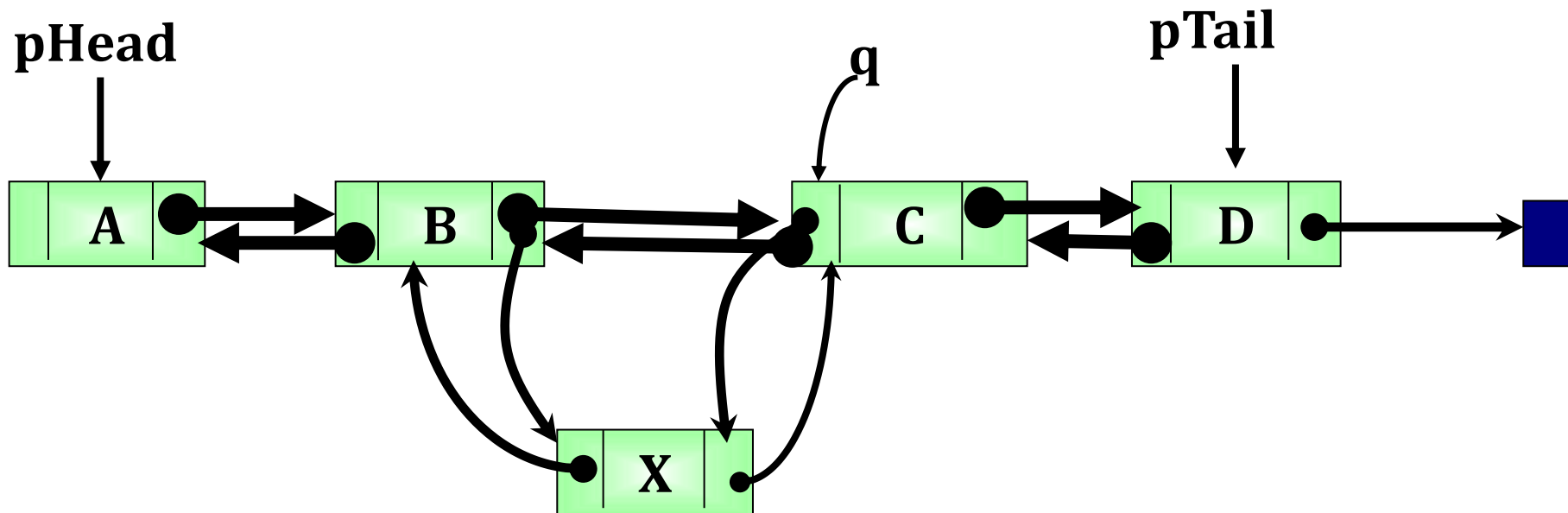
- Minh họa thêm nút X vào sau nút q



2. Danh sách liên kết

Thêm 1 nút vào trước nút q

- Minh họa thêm 1 nút vào trước nút q





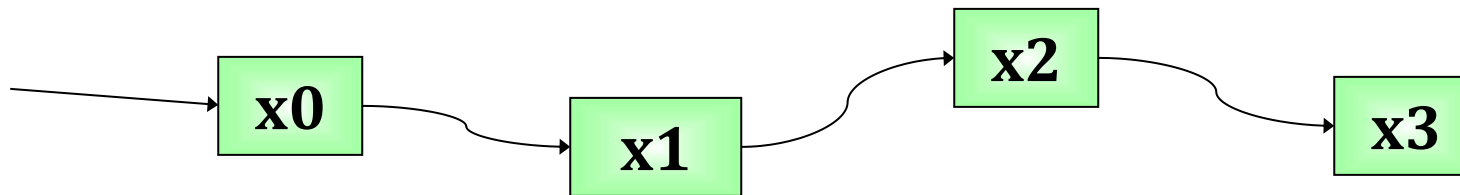
2. Danh sách liên kết

Danh sách liên kết đơn

- Gồm nhiều phần tử có cùng kiểu, kích thước.
- Số phần tử không cần xác định trước.
- Các phần tử được lưu trữ không nhất thiết liên kề nhau.
- Thường một con trỏ Head sẽ dùng để lưu trữ địa chỉ phần tử đầu.

2. Danh sách liên kết

- Tập hợp các node thông tin (khối dữ liệu) được tổ chức rời rạc trong bộ nhớ. Trong đó, mỗi node gồm hai phần:
 - **Thành phần dữ liệu:** dung để lưu thông tin của node
 - **Thành phần mối liên kết:** lưu trữ địa chỉ của phần tử kế tiếp trong danh sách, hoặc lưu trữ giá trị null nếu là phần tử cuối danh sách

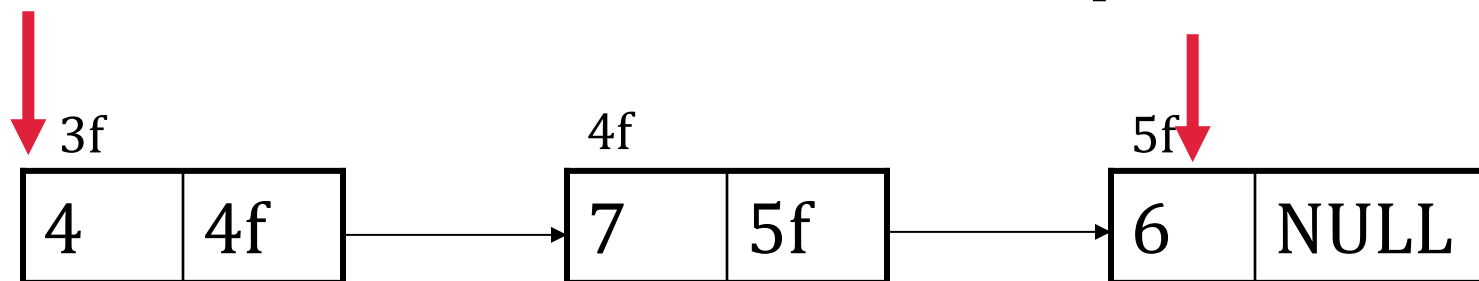


2. Danh sách liên kết

Ví dụ tổ chức DSLK đơn trong bộ nhớ

pHead

pTail



Trong ví dụ trên thành phần dữ liệu là 1 số nguyên



2. Danh sách liên kết

Các thao tác cơ bản trên DSLK đơn

- Tạo 1 danh sách liên kết đơn rỗng
- Tạo 1 nút có trường Info bằng x
- Tìm một phần tử có Info bằng x
- Thêm một phần tử có khóa x vào danh sách
- Hủy một phần tử trong danh sách
- Duyệt danh sách
- Sắp xếp danh sách liên kết đơn



2. Danh sách liên kết

Thêm một phần tử vào DSLK đơn

- Nguyên tắc thêm: Khi thêm 1 phần tử vào List thì có làm cho pHead, pTail thay đổi?
- Các vị trí cần thêm 1 phần tử vào List:
 - Thêm vào đầu List
 - Thêm vào cuối List
 - Thêm vào sau 1 phần tử q trong list



2. Danh sách liên kết

Thêm một phần tử vào DSLK đơn

➤ Thêm nút p vào đầu danh sách liên kết đơn

Bắt đầu:

Nếu List rỗng thì

+ pHead = p;

+ pTail = pHead;

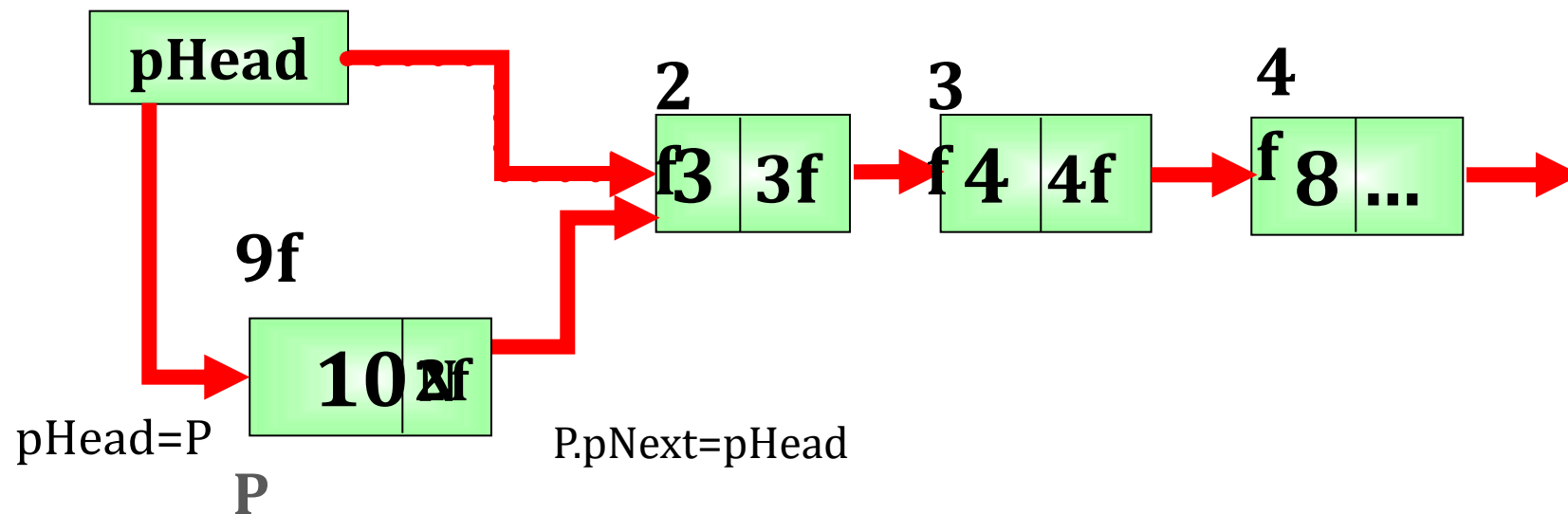
Ngược lại

+ p.pNext = pHead;

+ pHead = p

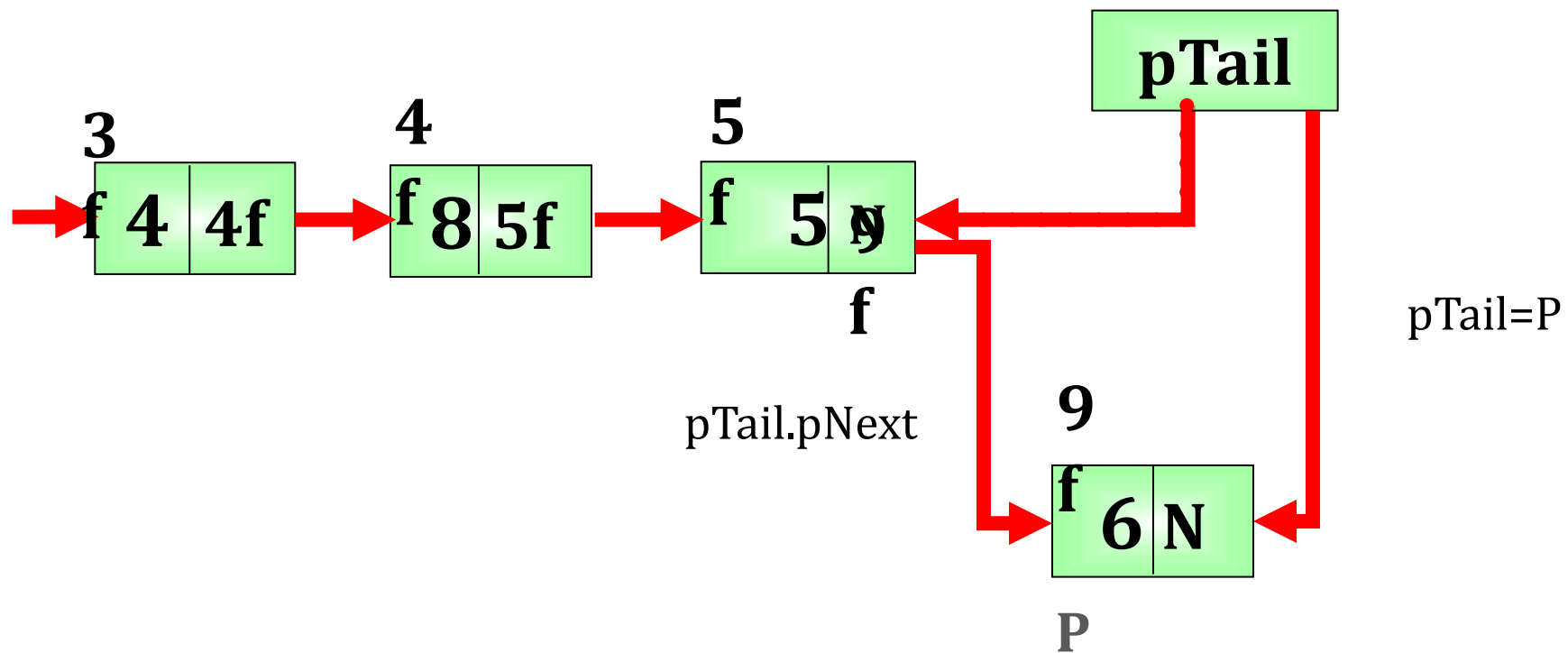
2. Danh sách liên kết

Minh họa thuật toán thêm vào đầu



2. Danh sách liên kết

Minh họa thuật toán thêm vào cuối





2. Danh sách liên kết

Hủy phần tử trong DSLK đơn

- **Nguyên tắc:** Phải cô lập phần tử cần hủy trước hủy.
- Các vị trí cần hủy:
 - Hủy phần tử đứng đầu List
 - Hủy phần tử có khoá bằng x
 - Hủy phần tử đứng sau q trong danh sách liên kết đơn
- Ở phần trên, các phần tử trong DSLK đơn được cấp phát vùng nhớ động bằng hàm new, thì sẽ được giải phóng vùng nhớ bằng hàm delete.



2. Danh sách liên kết

Thuật toán hủy phần tử trong DSLK đơn

Bắt đầu:

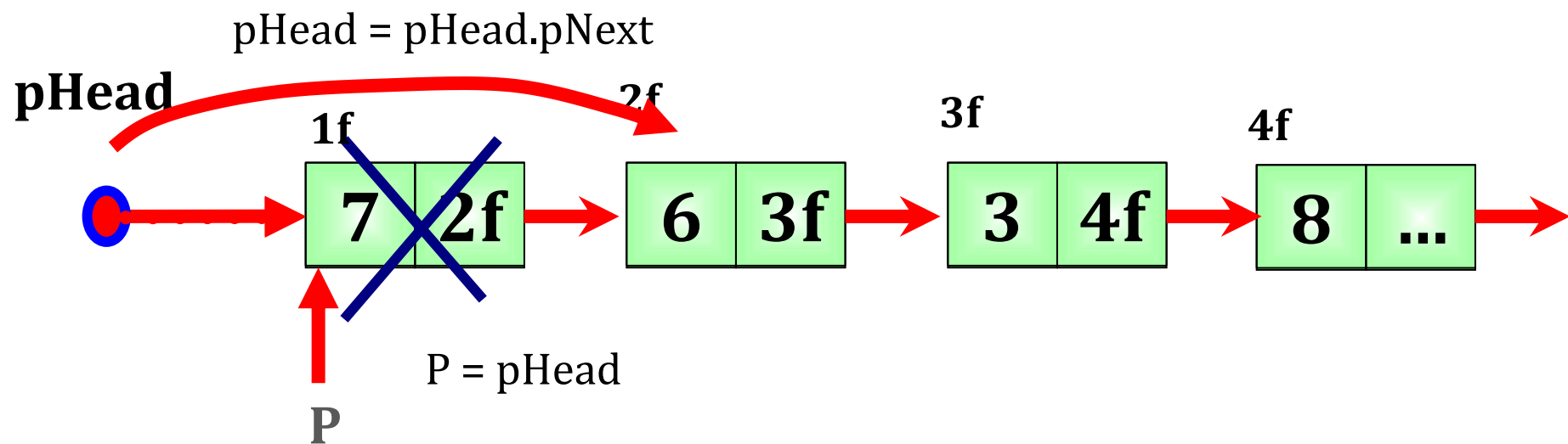
Nếu ($pHead \neq NULL$) thì

- B1: $p = pHead$ // p là phần tử cần hủy
- B2:
 - + $pHead = pHead.pNext$ // tách p ra khỏi ds
 - + $p = null$ // Hủy biến động do p trở đến
- B3:

Nếu $pHead == NULL$ thì $pTail = NULL$

2. Danh sách liên kết

Minh họa





2. Danh sách liên kết

Hủy phần tử sau phần tử q trong List

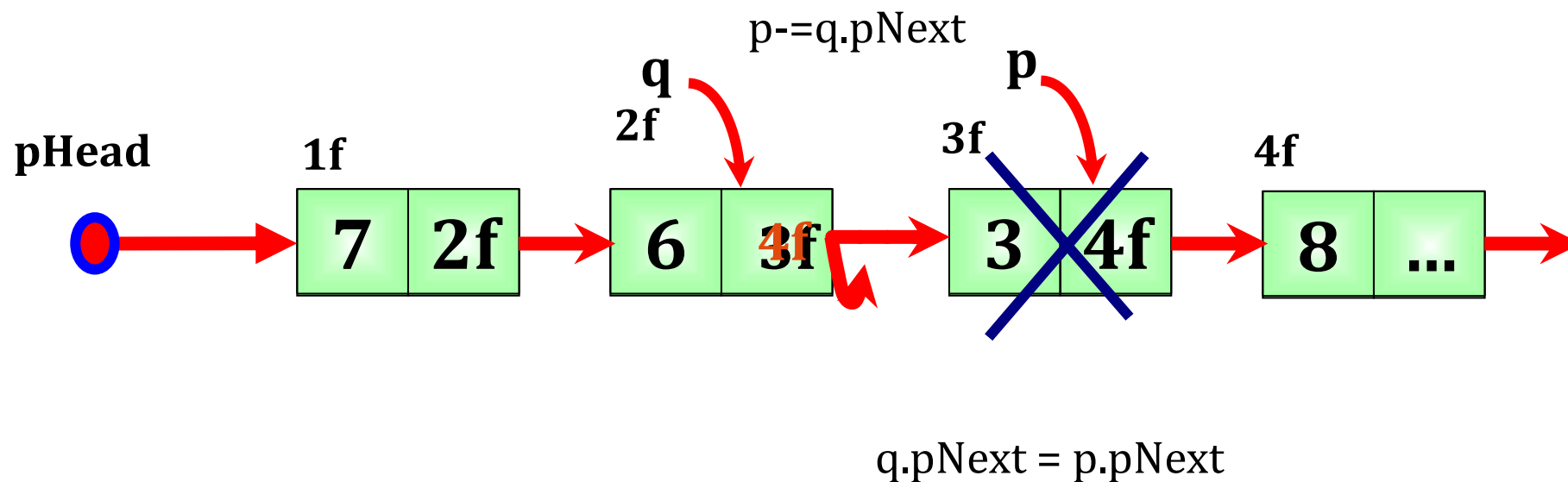
Bắt đầu

Nếu ($q \neq \text{null}$) thì // q tồn tại trong List

- B1: $p = q.pNext$; // p là phần tử cần hủy
- B2: Nếu ($p \neq \text{null}$) thì // q không phải là phần tử cuối
 - + $q.pNext = p.pNext$; // tách p ra khỏi xâu
 - + nếu ($p == pTail$) // nút cần hủy là nút cuối
 - $pTail = q$;
 - + $p = \text{null}$; // hủy p

2. Danh sách liên kết

Minh họa





2. Danh sách liên kết

Thuật toán hủy phần tử có khóa x

Bước 1:

Tìm phần tử p có khoá bằng x, và q đứng trước p

Bước 2:

Nếu (p != NULL) thì //tìm thấy phần tử có khoá bằng x

Hủy p ra khỏi List bằng cách hủy phần tử đứng sau q

Ngược lại

Báo không tìm thấy phần tử có khoá



2. Danh sách liên kết

Tìm 1 phần tử trong DSLK đơn

➤ Tìm tuần tự (hàm trả về), các bước của thuật toán tìm nút có Info bằng x trong list đơn

Bước 1: $p = pHead$; // địa chỉ của phần tử đầu trong list đơn

Bước 2:

Trong khi $p \neq NULL$ và $p.Info \neq x$

$p = p.pNext$; // xét phần tử kế

Bước 3:

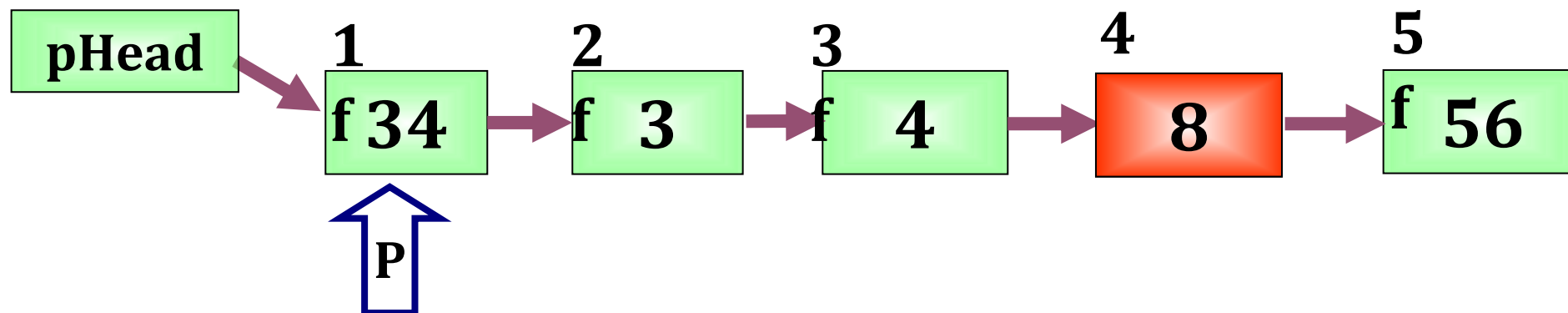
+ Nếu $p \neq NULL$ thì p lưu địa chỉ của nút có

$Info = x$

+ Ngược lại : Không có phần tử cần tìm

2. Danh sách liên kết

Minh họa



X = 8

Tìm thấy, hàm trả về địa chỉ của nút tìm thấy là 4f



2. Danh sách liên kết

Duyệt danh sách

➤ Duyệt danh sách là thao tác thường được thực hiện khi có nhu cầu cần xử lý các phần tử trong danh sách như:

- Đếm các phần tử trong danh sách
- Tìm tất cả các phần tử trong danh sách thỏa điều kiện
- Hủy toàn bộ danh sách



2. Danh sách liên kết

Thuật toán duyệt danh sách

Bước 1:

$p = pHead; //$ p lưu địa chỉ của phần tử đầu trong List

Bước 2:

Trong khi (danh sách chưa hết) thực hiện

+ xử lý phần tử p

+ $p = p.pNext; //$ qua phần tử kế



2. Danh sách liên kết

Hủy danh sách liên kết đơn

■ Bước 1:

Trong khi (danh sách chưa hết) thực hiện

- B11:

`p = pHead;`

`pHead = pHead.pNext; // cập nhật pHead`

- B12:

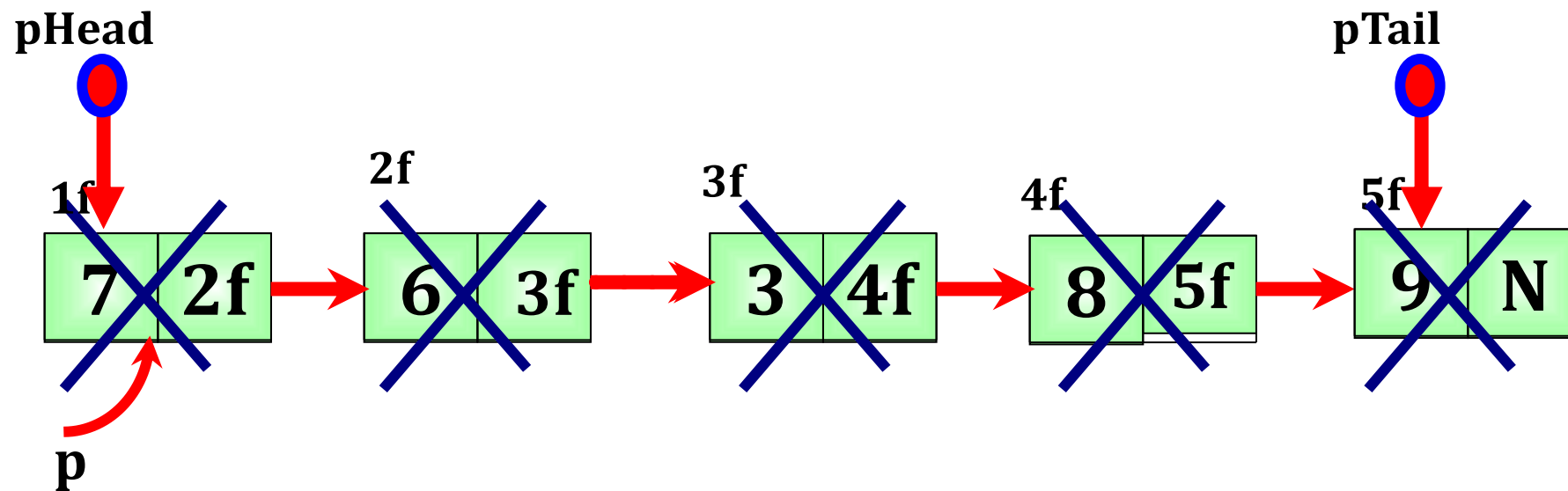
Hủy p

■ Bước 2:

`pTail = NULL; // bảo toàn tính nhất quán khi xâu rỗng`

2. Danh sách liên kết

Minh họa



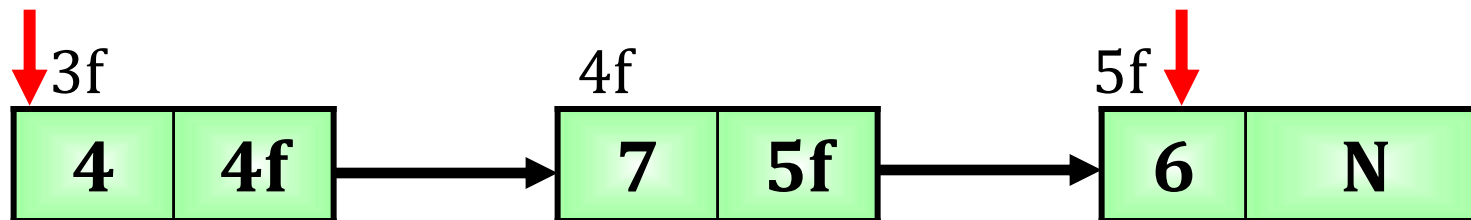
2. Danh sách liên kết

Sắp xếp danh sách

- Có hai cách tiếp cận
- **Cách 1:** Thay đổi thành phần Info

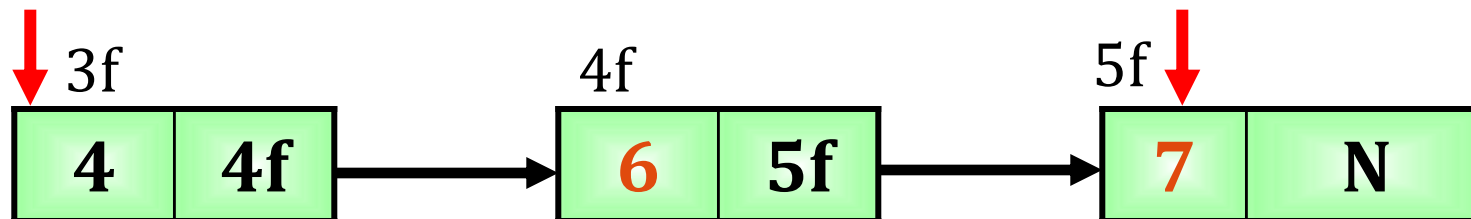
pHead

pTail



pHead

pTail



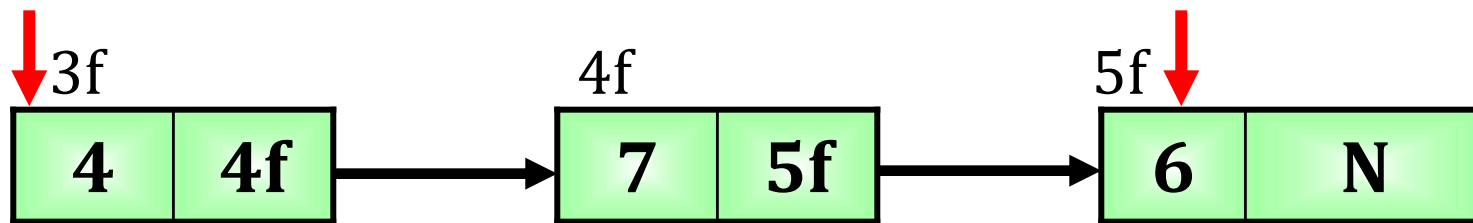
2. Danh sách liên kết

Sắp xếp danh sách

➤ **Cách 2:** Thay đổi thành phần pNext (thay đổi trình tự móc nối của các phần tử sao cho tạo lập nên được thứ tự mong muốn)

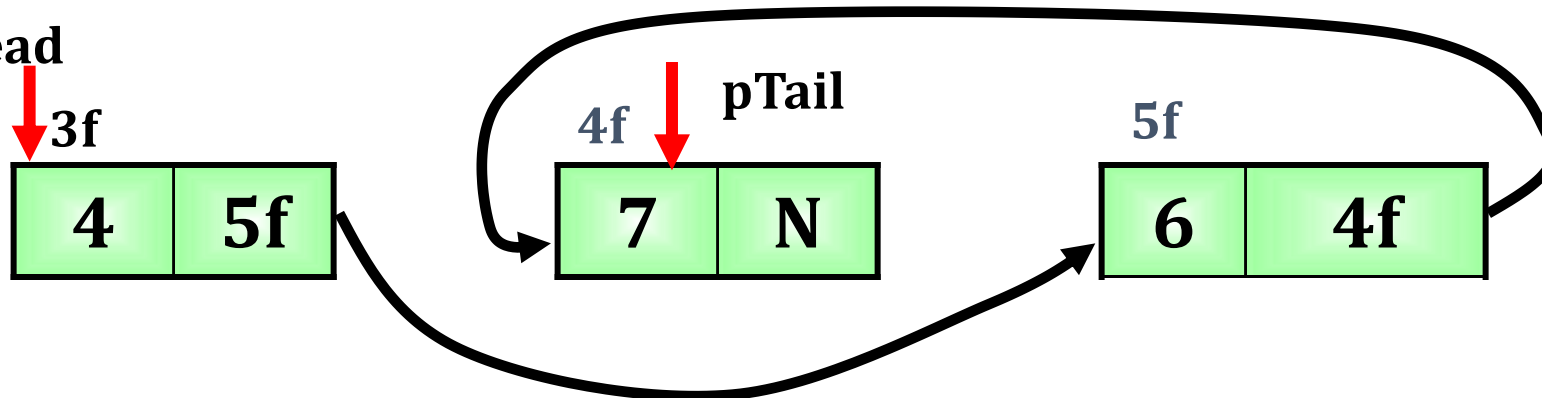
pHead

pTail



pHead

pTail





2. Danh sách liên kết

Ưu và nhược điểm

➤ Thay đổi thành phần Info (dữ liệu)

- Ưu: Cài đặt đơn giản, tương tự như sắp xếp mảng

- Nhược:

Đòi hỏi thêm vùng nhớ khi hoán vị nội dung của 2 phần tử -

> chỉ phù hợp với những cấu trúc dữ liệu có kích thước Info nhỏ

Khi kích thước Info (dữ liệu) lớn chi phí cho việc hoán vị thành phần Info lớn

✓ Làm cho thao tác sắp xếp chậm

➤ Thay đổi thành phần pNext

- Ưu:

Kích thước của trường này không thay đổi, do đó không phụ thuộc vào kích thước bản chất dữ liệu lưu tại mỗi nút.

✓ Thao tác sắp xếp nhanh

- Nhược: Cài đặt phức tạp



2. Danh sách liên kết

Cài đặt hàm main()

- Yêu cầu: Viết chương trình thành lập 1 xâu đơn, trong đó thành phần dữ liệu của mỗi nút là 1 số nguyên dương.
- 1. Liệt kê tất thành phần dữ liệu của tất cả các nút trong xâu
- 2. Tìm 1 phần tử có khoá bằng x trong xâu.
- 3. Xoá 1 phần tử đầu xâu
- 4. Xoá 1 phần tử có khoá bằng x trong xâu
- 5. Sắp xếp xâu tăng dần theo thành phần dữ liệu (Info)
- 6. Chèn 1 phần tử vào xâu, sao cho sau khi chèn xâu vẫn tăng dần theo trường dữ liệu



2. Danh sách liên kết

Vài ứng dụng danh sách liên kết đơn

- Dùng xâu đơn để lưu trữ danh sách các học viên trong lớp học
- Dùng xâu đơn để quản lý danh sách nhân viên trong một công ty, trong cơ quan
- Dùng xâu đơn để quản lý danh sách các cuốn sách trong thư viện
- Dùng xâu đơn để quản lý các băng đĩa trong tiệm cho thuê đĩa.



2. Danh sách liên kết

Dùng sâu đơn để quản lý lớp học

➤Yêu cầu: Thông tin của một sinh viên gồm, mã số sinh viên, tên sinh viên, điểm trung bình.

1. Hãy khai báo cấu trúc dữ liệu dạng danh sách liên kết để lưu danh sách sinh viên nói trên.
2. Nhập danh sách các sinh viên, và thêm từng sinh viên vào đầu danh sách (việc nhập kết thúc khi tên của một sinh viên bằng rỗng)
3. Tìm một sinh viên có trong lớp học hay không
4. Xoá một sinh viên có mã số bằng x (x nhập từ bàn phím)
5. Liệt kê thông tin của các sinh viên có điểm trung bình lớn hơn hay bằng 5.
6. Xếp loại và in ra thông tin của từng sinh viên, biết rằng cách xếp loại như sau:
 - ĐTB ≤ 3.6 : Loại yếu
 - ĐTB ≥ 5.0 và ĐTB < 6.5 : Loại trung bình
 - ĐTB ≥ 6.5 và ĐTB < 7.0 : Loại trung bình khá
 - ĐTB ≥ 7.0 và ĐTB < 8.0 : Loại khá
 - ĐTB ≥ 8.0 và ĐTB < 9.0 : Loại giỏi.
 - ĐTB ≥ 9.0 : Loại xuất sắc
7. Sắp xếp và in ra danh sách sinh viên tăng theo điểm trung bình.
8. Chèn một sinh viên vào danh sách sinh viên tăng theo điểm trung bình nói trên, sao cho sau khi chèn danh sách sinh viên vẫn tăng theo điểm trung bình

THỰC HÀNH

#thực hành 1.1:

#Array List

```
thislist = ["hanoi", "saigon", "danang", "nhatrang", "cantho"]  
print(thislist)
```

```
thislist.insert(2, "cantho")    #insert  
print(thislist)  
thislist.insert(3, "Hue")  
print(thislist)
```

```
thislist = ["hanoi", "saigon", "danang", "nhatrang"]  
thislist.pop(2)                #pop
```

```
thislist = ["hanoi", "saigon", "danang", "nhatrang", "cantho"]  
thislist.sort()  
print(thislist)
```

```
thislist = [100, 15, 50, 65, 82, 23]  
thislist.sort()                #sort  
print(thislist)
```

```
list1 = ['data analytics', 'data science', 'data structures and  
algorithms', 2020, 2021]  
list2 = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
print ("list1[1]: ", list1[1])  
print ("list2[2:8]: ", list2[2:8])
```

THỰC HÀNH

#thực hành 1.2:

```
List1 = [0, 2, 4, 10, 30, 15, 25, 60, 50]  
print("\nDanh sach cac so ")  
print(List1)
```

```
List2 = [4, 4, 'Data Science', 5, 'For', 10, 'Machine Learning']  
print("\nDanh sách voi các giá trị hỗn hợp")  
print(List2)
```



THỰC HÀNH

#thực hành 1.3:

#Array List

```
arr= [37, 12, 5, 100, 11, 10, 15, 55, 80, 25]
```

```
min_arr = arr[9]
```

```
for i in arr:
```

```
    if (i < min_arr):
```

```
        min_arr = i
```

```
print('min value:', min_arr)
```

THỰC HÀNH

#thực hành 2.1:

```
#Linked List;
```

```
class Node:
```

```
    def __init__(self, data):  
        self.data = data    # data  
        self.next = None    # Initialize next as null
```

```
class LinkedList:
```

```
    def __init__(self):  
        self.head = None
```

```
    def printList(self):  
        temp = self.head  
        while (temp):  
            print (temp.data)  
            temp = temp.next
```

```
if __name__=='__main__':
```

```
    llist = LinkedList()  
    llist.head = Node(10)  
    second = Node(11)  
    third = Node(12)
```

```
    llist.head.next = second;    # Link first with second  
    second.next = third;        # Link second with the third  
    llist.printList()
```

THỰC HÀNH

#thực hành 2.2:

#Linked List

```
class Node:
```

```
    def __init__(self, data):  
        self.data = data  
        self.next = None
```

```
def search(head):  
    currentNode = head  
    while currentNode:  
        print(currentNode.data, end=" -> ")  
        currentNode = currentNode.next  
    print("null")
```

```
def delete_node(head, nodeToDelete):  
    if head == nodeToDelete:  
        return head.next
```

```
    currentNode = head  
    while currentNode.next and currentNode.next != nodeToDelete:  
        currentNode = currentNode.next
```

```
    if currentNode.next is None:  
        return head  
    currentNode.next = currentNode.next.next  
    return head
```

```
node1 = Node('AI')  
node2 = Node('ML')  
node3 = Node('DL')  
node4 = Node('IT')  
node5 = Node('Blockchain')
```

```
node1.next = node2  
node2.next = node3  
node3.next = node4  
node4.next = node5
```

```
print("Before del:")  
search(node1)  
# Delete node4  
node1 = delete_node(node1, node4)  
print("\nAfter del:")  
search(node1)
```


THỰC HÀNH

#thực hành 2.3:

```
#Linked_list
from collections import deque
deque()
deque([{'data': 'a'}, {'data': 'b'}])
l1ist = deque("abcd")      #??
l1ist
l1ist.append("h")          #??
l1ist
l1ist.pop()               #??
l1ist                      #??

from collections import deque
d = deque([1,2,3,4,5,6])
print(d)
for i in d:
    print(i)

print(d.pop(), d)         #??
```



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

